

生成AIによるソフトウェア開発

要求定義・設計・実装・テスト

本日の内容

1. **要求定義** - 生成AIによる要求工学プロセスの支援
2. **設計** - 設計工程への生成AI適用
3. **実装** - コード生成とプロンプトエンジニアリング
4. **テスト** - テスト仕様書・テストコードの生成

1. 要求定義

要求工学とは

顧客から要求を獲得し、それを要求仕様として記述する技術分野（REBOK）

関連知識	内容
要求獲得	ステークスホルダから会議やインタビューで要求を獲得
要求分析	要求を整理し、優先順位付けを行い、実現すべき要求を絞る
要求仕様化	分析された要求を規定の書式で要求仕様書として記述
要求の検証	要求仕様書の正当性・妥当性を確認

生成AIによる要求工学プロセスの支援

ペルソナパターンの活用

複数の専門家ペルソナを同時に演じさせる

あなたは以下のペルソナを同時に演じ、矛盾があれば「意思決定が必要」と明示して論点整理する。

- プロダクトマネージャ（PM）
- コンタクトセンター設計者
- テレフォニー/VoIPエンジニア
- セキュリティ & 法務担当
- UX設計者

生成AIへの指示構成

項目	内容
背景・課題	解決したい問題の説明
目的（ゴール）	達成したい状態
前提・制約	不明点は「要確認事項」として列挙
機能要件	ユースケース→画面→データ→API→例外まで落とし込む
非機能要件	可用性、セキュリティ、運用、コンプライアンス
出力フォーマット	PRD、ユーザーストーリー、データモデル等

生成AIの出力例

要確認事項リスト（優先度つき）

最優先（P0：決まらないと設計が止まる）

- 回線取り込み方式（SIP連携可否、PBX/ゲートウェイ）
- 録音告知の必須要件（法務/社内規程）

重要（P1：MVPの仕様がブレる）

- 重要顧客の定義（ホワイトリスト運用か、CRMか）
- 権限設計（誰がDLできるか、監査要件）

ユースケースへの展開

UC-01 着信を番号別ルールで判定する

目的：着信ごとに「拒否/居留守/アナウンス/IVR/通常着信」を自動決定

主アクター：システム

トリガー：ひかり電話に着信が入る

基本フロー：

1. 発信者番号を取得（非通知の場合は匿名扱い）
2. 電話番号を正規化
3. ルールを優先順位順に評価
4. 採用したアクションを実行

代替フロー：

A1：番号が取得できない→匿名ルールへ

IEEE標準フォーマットへの変換

企業独自フォーマットや標準形式への変換も可能

この要求仕様、ユースケースをIEEE標準フォーマットに出力してください。

→ IEEE830構成のSRS（Software Requirements Specification）を生成

2. 設計

設計工程への生成AI適用方針

ソフトウェア設計とは

「何を解くか」を定めた要求仕様をもとに、「どのようにして解くか」まで具体化し、実装工程へのスムーズな移行を目指す

方針

人間が主な作業者となり、一部の作業を生成AIに支援させる

1. 支援・代替させる作業に必要な文脈情報を
2. 他の不要な情報を省きつつ
3. いかに正確に理解させるか

設計工程に必要な入力文書

文書	新規開発	流用開発
要求仕様書(新版)	○	○
組織内の標準・規定文書	○	○
設計書(執筆済み部分)	○	○
設計書(旧版)		○
ソースコード(旧版)		○
過去の類似プロジェクトのデータ	○	○

生成AIが読みやすい設計書

テキストマークアップ言語を使用

- **PlantUML / Mermaid形式** で図を記述
- 表：行・列の見出しとデータ部分を明確に
- CSV → JSON に変換すると可読性向上

複雑な文書の場合

- データの前処理を行う
- 読み込み方の指示を与える

表データの「テキストとしての自然さ」は、続く応答の品質に影響する

設計書のドラフト作成手順

1. **工程定義** - 生成AI自身に要求～詳細設計までの各工程の説明・定義を行わせる
2. **雛形作成** - 工程情報とシステム概要を与え、基本設計書の雛形・章・節を提案させる
3. **章ごとに生成** - 設計書の雛形と要求定義書を与えて、各章を順序よく生成
4. **ブラッシュアップ** - 作成した設計書を改善させる

生成AIによるレビュー

できること

- あいまいな表現の指摘
- 矛盾の指摘
- 考慮できていない暗黙の条件の発見

指示例

以下の設計書をレビューしてください。

- あいまいな表現を指摘してください
- 矛盾点があれば指摘してください
- 暗黙の前提条件があれば明示してください

設計工程の今後

現状の課題

- 業種、業界、組織などのドメイン知識の影響が強い
- 設計者の経験知、暗黙知が重要

今後の展開

- 設計工程の完全自動化はまだまだ先
- 人間に逆質問するような仕組みで改善していく方向

3. 実装

プログラムの実装とは

要求の詳細化を踏まえた設計の意図を正しく理解し、後からでも万人にも読みやすく、保守しやすい形で具体化する創造的かつ論理的なプロセス

重要なポイント

- 変数や関数のわかりやすい命名
- 処理の粒度や使用頻度に応じた適切な関数分割
- 無駄のない制御構造
- 必要に応じたコメント

読みやすさが最重要。他人が理解しやすいコードを書くことは、正確さや効率性と並ぶほどに重要な品質

生成AIの活用が有効なタスク

タスク	説明
コードの生成・補完	仕様からコードを生成
コメント生成	コードからコメントを自動生成
API/ライブラリの利用方法提示	使い方を説明
コードレビュー	問題点の指摘
デバッグ支援	エラーの原因特定
リファクタリング支援	コード改善提案
テストケース/テストコード生成	テストを自動生成

プロンプトエンジニアリング

(1) ゼロショットプロンプティング

〇〇のコードを書いて

- 長所：一般的なタスクには機能する
- 短所：仕様の細かい部分を理解しきれないことがある

(2) フューショットプロンプティング

- 例を提示してスタイルに合わせた生成
- 短所：例の品質に依存

(3) Chain of Thought (CoT)

- 段階的に考えさせる

コード生成のプロンプト構成

CoTを用いて以下の構成でプロンプトを設計

- ① 指示文書
 - 何を生成するか、どのような形式か
- ② 規約
 - コーディング規約、命名規則
- ③ 設計書
 - 詳細仕様（Markdown形式推奨）

Markdown形式は学習データに多く使われているため効果的

ハルシネーション対策

ハルシネーションとは

- 内在的 - 入力と矛盾する出力
- 外在的 - 入力から検証できない出力

対策

- 目的にあったLLMの選択
- プロンプトの最適化
- RAGの構築（設計ドキュメント、技術仕様書をDB化）
- ファインチューニング

コードレビューの重要性

フィードバックループ

プロンプト設計・最適化



コードの自動生成



人間によるコードレビュー



(問題があれば戻る)

- コードレビューは必ず行い、プロンプトと一致していることを確認
- 品質が悪い場合、入力量が多いことも原因
- 機能細分化、設計情報の簡素化、段階的生成で対策

LLMによるコード生成の未来像

人間の役割は残り続ける

- 過去に存在しないオリジナルかつクリエイティブな高品質コードを作るのは人間
- 実装のアイデア・品質がソフトウェア全体の満足度・信頼性・保守性に直結

必要な人材

- 独自性、創造性を持つ
- 品質に高い関心を持つ
- 生成AI・周辺ツールの特徴を理解する

望ましいのは、人間と生成AIとの協業

4. テスト

テスト仕様書の作成ステップ

Step	内容
1	入力となるテストベース（ドキュメント、ソースコード）を準備
2	テストベースからの仕様把握、目的、テストアイテムの明確化
3	テスト観点、テスト条件の抽出
4	テストケース（概要）の作成
5	テストケース（詳細）の作成
6	テスト仕様書の最終レビューと整合性確認

Step 1-2: テストベースの準備と仕様把握

入力となるテストベース

- 要求仕様書
- 基本設計書/詳細設計書
- ソースコード
- 過去の障害票

ドキュメントはMarkdown形式やPlantUML形式へ変換を推奨

ID体系

- テストアイテム: ITEM-XXX
- 要求・リスク: REQ/NF/RISK-XXX

Step 3: テスト観点の抽出

- ITEM/REQ/RISKを参照し、正常/異常/境界値/非機能要件等のテスト観点を洗い出して連番(C-XXX)付与
- 全要求・リスクが観点と紐付いているかAIでチェック

出力形式

観点ID	ITEM-ID	REQ/RISK-ID	観点概要
C-001	ITEM-001	REQ-001	入力バリデーション

差分チェック例

Step1のREQ-001, NF-001, RISK-001に対して
C-001, C-002...で未対応は？

Step 4-5: テストケースの作成

概要 (G-XXX)

G-001 | C-001 | ITEM-001 | パスワード桁数のエラー表示 | 優先度: 高

詳細 (D-XXX)

D-001 | G-001 |
前提: ログイン画面 |
操作: パス4文字入力→送信 |
入力: abcd |
期待結果: 正常ログイン |
優先度: 高

Step 6: 最終レビューと整合性確認

テスト仕様書の構成

1. はじめに
2. テストアイテム・要求/リスク一覧
3. 観点一覧
4. ケース（概要）
5. ケース（詳細）
6. トレーサビリティ（REQ→C→G→D）
7. 指摘・修正履歴

AIによる差分チェック

要求/リスク一覧 vs 詳細ケース一覧で未カバーIDは？

テストコード生成のプロンプト

Step 1: コードの理解と要約

[目的] 対象コードの目的、動作、エッジケースを把握

[出力形式]

- 役割・処理内容：<簡潔な説明>
- 入出力：<説明>
- 特殊な挙動・エッジケース：<説明>

Step 2: テストケースの網羅的な列挙

[目的] 正常系、異常系、境界値など全ケースを洗い出す

[出力形式]

- 正常系：ケース1:<説明>
- 異常系・エッジケース：ケース2:<説明>

テストコード生成のプロンプト（続き）

Step 3: テストコードの生成

[目的] 網羅性の高いユニットテストコードを生成

[指示]

- 前ステップのテストケースを全てカバー
- 同種のテストはパラメタライズ化して簡潔に記述
- テスト関数名は目的が明確になるようにつける

[入力]

- ステップ2のテストケース
- 対象コードのシグネチャ

まとめ

各工程における生成AIの役割

工程	生成AIの役割	人間の役割
要求定義	詳細化、ユースケース展開、フォーマット変換	意思決定、優先順位付け
設計	レビュー、ドラフト作成、図の生成	設計判断、暗黙知の反映
実装	コード生成、補完、リファクタリング	レビュー、創造的なコード
テスト	仕様書作成、テストコード生成	観点の網羅性確認

生成AI活用の共通原則

1. **人間が主体、AIは支援** - 最終判断は人間が行う
2. **入力の品質が出力を決める** - 明確な仕様・指示が重要
3. **レビューは必須** - ハルシネーション対策
4. **段階的に進める** - 一度に全てを生成させない
5. **フィードバックループ** - 出力を確認し、プロンプトを改善

参考文献

生成AIによるソフトウェア開発

設計からテスト、マネジメントまでをすべて変革するLLM活用の実践体系
(森北出版)